

OPERATING SYSTEMS • PROCESS MANAGEMENT

Linux Processes

The process tree, `fork()`, `exec()`, and `wait()` — how one program becomes many

`fork()`

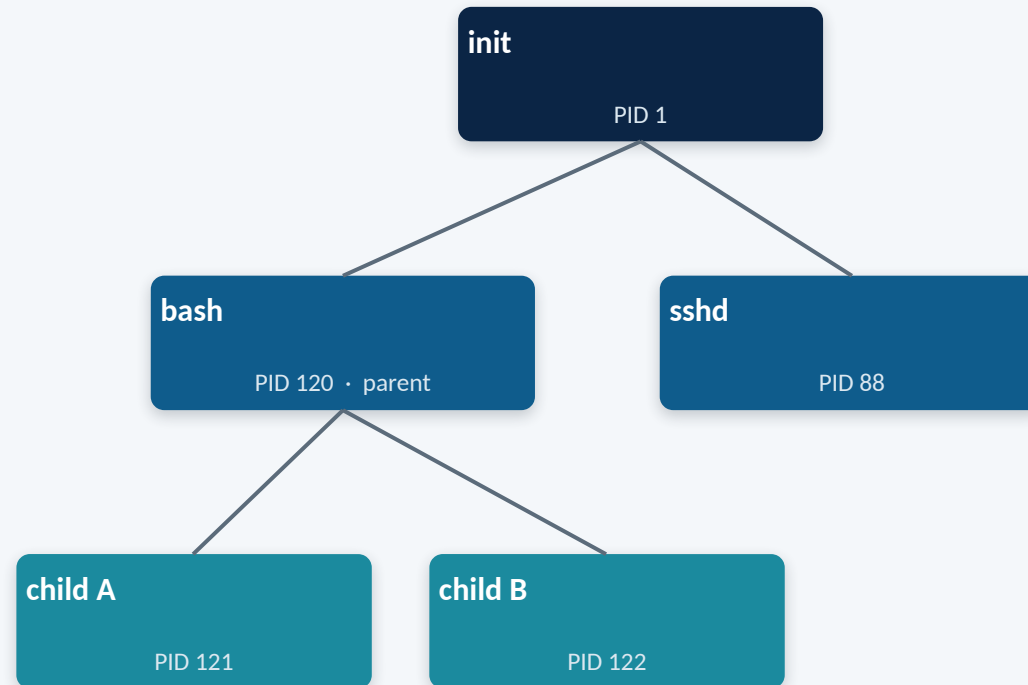
`getpid()/getppid()`

`execlp()`

`wait() / waitpid()`

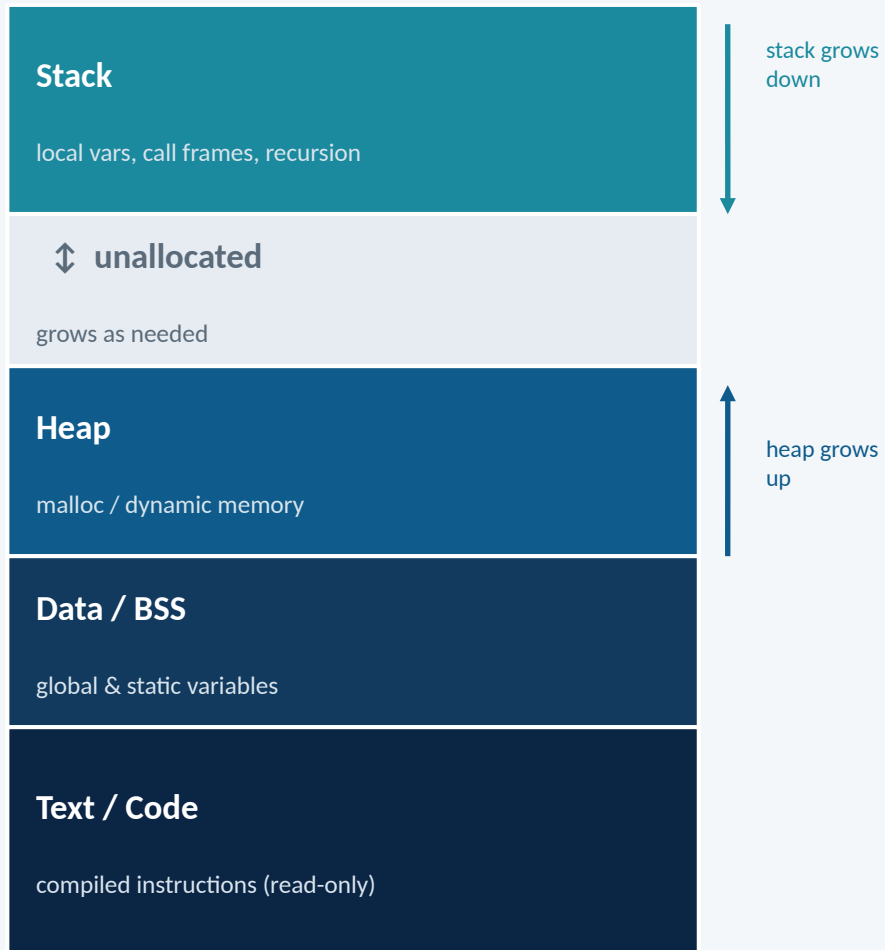
Every Process Has a Parent

- Linux arranges all running processes into a single tree, rooted at init (PID 1).
- `fork()` creates a new process — the child — as a copy of the calling process, the parent.
- `getpid()` returns a process's own ID; `getppid()` returns its parent's ID.
- If a parent exits before its child, the child is orphaned and gets adopted by init.



`getppid()` of child A → **120**
`getpid()` of bash → **120**

A Process's Memory Space



Code / Text segment

The compiled machine instructions of the program. Shared and read-only.

Data & BSS

Global and static variables — initialized (Data) and uninitialized (BSS).

Heap

Dynamically allocated memory the program requests at runtime (malloc/free).

Stack

Local variables and function call frames — including frames for recursive calls.

How fork() Works

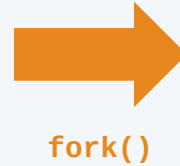
BEFORE THE CALL

```
int main(void) {  
    pid_t pid = fork();  // <-- snapshot  
    ...  
}
```

compiled binary running as a single process

process

PID 500



AFTER THE CALL

parent

PID 500

child (new)

PID 501

parent

PID 500

both processes now
continue right after fork()

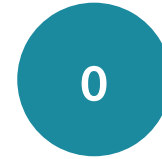
fork() clones the entire process — code, data, heap, and stack — into a brand-new address space. Execution resumes for both processes at the instruction right after the fork() call.

Telling Parent and Child Apart

```
pid_t pid = fork();

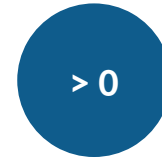
if (pid == 0) {
    // child process
} else if (pid > 0) {
    // parent; pid = child's PID
} else {
    // fork failed, no child made
}
```

The single `fork()` call returns twice — once in each process — with a different value each time. That return value is the only way to tell them apart.



Child

`fork()` succeeded — this code is running inside the new child process.



Parent

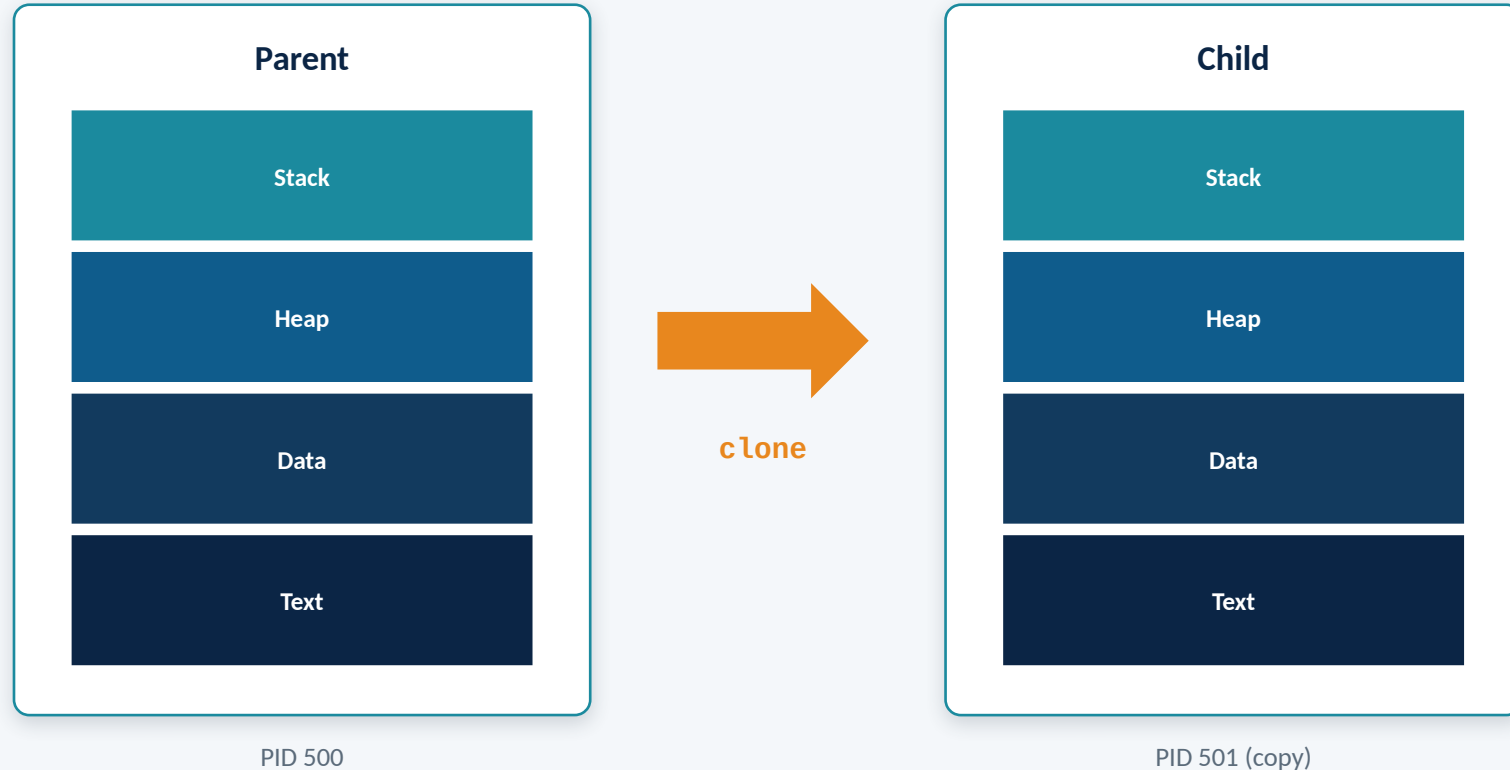
`fork()` succeeded — the value is the PID of the newly created child.



Parent only

`fork()` failed (e.g. resource limits) — no child was created.

fork() Duplicates the Memory Space



What gets copied

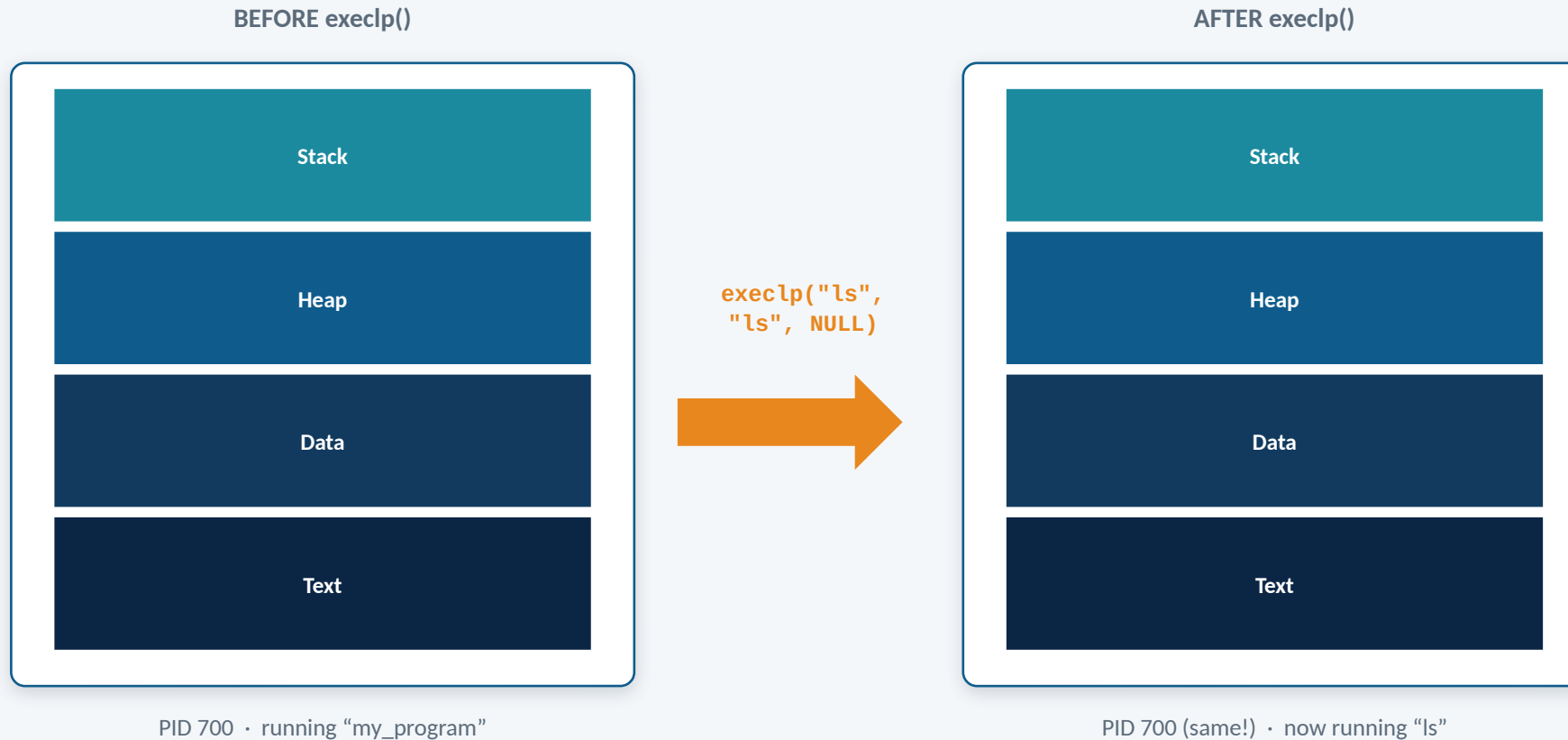
- Code, data, heap and stack segments
- Open file descriptors
- Program counter / execution point

What differs

- Process ID (PID) and parent PID
- `fork()`'s return value

Real kernels use copy-on-write: pages are shared until one process writes to them — the full-copy picture is the logical model, not the literal implementation.

exec() Replaces the Process Image



- Same PID and process table entry — only the program image changes.
- Code, data, heap and stack are completely overwritten.
- On success, `exec()` never returns — the old code is gone.
- Common pattern: `fork()` then `exec()` in the child.

Where `fork()`, `exec()` and `wait()` Come From



ISO C Standard

Portable across any C compiler / OS: `printf()`, `malloc()`, `strlen()`.

POSIX

Defines the OS-facing interface: `fork()`, `execlp()`, `wait()`, `getpid()`, `getppid()`.

glibc

Linux's C library. Implements POSIX and ISO C, and adds Linux-specific extensions.

Reaping Children: wait() & waitpid()

- Ideally, a parent calls wait() or waitpid() and blocks until its child finishes.
- This lets the parent collect the child's exit status.
- It also lets the kernel free the child's process-table entry.
- If the parent never waits, cleanup and re-parenting still happen — just later.

```
pid_t child = fork();
if (child > 0) {
    int status;
    waitpid(child, &status, 0);
}
```



Parent waits — clean exit

Child finishes → parent's waitpid() unblocks, reads the exit status, and the kernel removes the child's entry immediately.



Zombie process

Child exits before the parent calls wait() — it becomes a zombie: gone from memory, but its exit status lingers until the parent reaps it.

Orphan process: if the parent exits first, the child is adopted by init (PID 1), which will call wait() on its behalf.

Putting It All Together



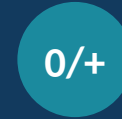
Process tree

Every process has a parent; the tree is rooted at init (PID 1).



fork()

Clones the calling process — code, data, heap, stack — into a new one.



Return value

0 in the child, child's PID in the parent, -1 on failure.



exec() family

Overwrites the current process image with a new program; PID stays.



wait()/waitpid()

Parent blocks for the child's exit status and lets the kernel reap it.



Zombies & orphans

Unreaped children linger as zombies; parentless children are adopted by init.